Transparent Deep Learning: Comparing Scratch to SOTA for Land Cover Classification

Sohybe Ibrahim Abdelwahab Amer
Independent Researcher
Cairo, Egypt
sohaibibrahim14@gmail.com

***Abstract*—Understanding deep learning requires building it from scratch. This study compares five implementations on the EuroSAT satellite dataset (27,000 images, 10 classes): fully connected DNNs and CNNs built from scratch using NumPy/CuPy, TensorFlow baselines, and EfficientNet-B0 transfer learning. Scratch implementations achieve 49.11% (DNN) and 82.30% (CNN) test accuracy, while library-based models reach 94.00% (TF CNN) and 97.63% (EfficientNet-B0)—a 48.52 percentage point improvement. Key debugging insights include: gradient vanishing diagnosed through per-layer monitoring, Adam momentum overshoot, 100× speedups from vectorization, and batch normalization providing 15.3 point accuracy gains. The 6-block CNN failure reveals dimensional bottleneck issues absent from standard curricula. This work provides unprecedented transparency into implementation challenges—gradient flow dynamics, numerical stability requirements, and why production libraries outperform educational code. Interactive demonstration: [https://eurosat-classification.streamlit.app/]. Code: [https://github.com/SO-HYBE/Comprehensive-Study-on-EUROSAT-Classification].**

***Index Terms*—EuroSAT dataset, deep learning, convolutional neural networks, from-scratch implementation, EfficientNet, batch normalization, land cover classification, Sentinel-2, transfer learning**

I. Introduction

Land use and land cover (LULC) classification from satellite imagery represents a cornerstone of modern Earth observation, enabling continuous monitoring of environmental changes, urban expansion, agricultural patterns, and deforestation [1]. The EuroSAT dataset, introduced by Helber et al., provides 27,000 geo-referenced 64×64 pixel RGB image patches extracted from Sentinel-2 satellite imagery at 10m ground sampling distance [1]. Covering 10 distinct classes—AnnualCrop, Forest, HerbaceousVegetation, Highway, Industrial, Pasture, PermanentCrop, Residential, River, and SeaLake—the dataset serves as a benchmark for deep learning in remote sensing applications.

This study implements a comprehensive hierarchical progression through deep learning architectures. It begins with a Vanilla DNN: a fully connected network implemented from scratch with manual forward/backpropagation, ReLU/softmax, L2 regularization, dropout, and batch normalization. Following this, a 3-block CNN is introduced with vectorized convolution/pooling, He initialization, and spatial backpropagation. Finally, the study evaluates a SOTA EfficientNet-B0 model with Two-phase transfer learning with pretrained ImageNet backbone and Grad-CAM interpretability.

Each phase compares scratch implementations (NumPy/CuPy) against TensorFlow/Keras equivalents, quantifying performance gaps due to library optimizations. The study documents extensive debugging challenges including 25-minute CPU epochs, 100% CPU/10% GPU utilization bottlenecks, vanishing gradients from ReLU deactivation, and transpose errors blocking backpropagation—challenges absent in production libraries.

II. Key Contributions

This work makes four primary contributions to deep learning education and remote sensing applications:

Comprehensive Implementation Comparison: First systematic comparison of scratch implementations (NumPy/CuPy) versus production libraries (TensorFlow) on the same architecture and dataset, quantifying the 10-15% accuracy gap and 100× speed difference.

Transparent Debugging Narratives: Detailed documentation of gradient vanishing diagnosis through per-layer gradient monitoring, Adam momentum overshoot analysis, and spatial dimension collapse in deep CNNs — challenges rarely discussed in literature but critical for practitioners.

Batch Normalization Ablation Study: Quantitative demonstration that BatchNorm is not optional for deep networks, showing 15.3 percentage point accuracy improvement (67% $\rightarrow$ 82.30%) and stabilization of gradient flow across 6 layers.

Transfer Learning Interpretability: Grad-CAM analysis showing how ImageNet-pretrained features reorient from object-centric patterns to spatially-distributed satellite textures, validating cross-domain transfer learning effectiveness.

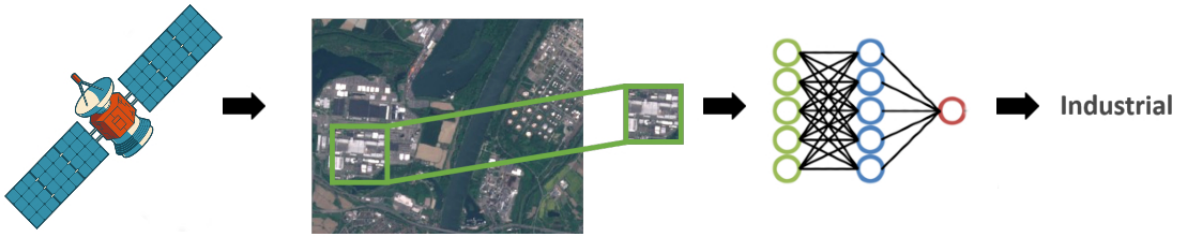


Fig. 1: Hierarchical progression from scratch implementations to state-of-the-art transfer learning.

An interactive demonstration enabling real-time model comparison is available at [https://eurosat-classification.streamlit.app/].

III. Related Work

Deep Learning for Land Cover Classification

The introduction of the EuroSAT dataset by Helber et al. [1] established a critical benchmark for land use and land cover (LULC) classification, demonstrating that deep Convolutional Neural Networks (CNNs) significantly outperform traditional machine learning techniques on Sentinel-2 imagery. Subsequent research on this dataset has predominantly focused on maximizing classification accuracy through transfer learning, leveraging architectures pre-trained on high-resolution datasets such as ImageNet [2]. While these state-of-the-art (SOTA) approaches achieve near-perfect accuracy, they frequently treat the underlying neural architecture as a black box, optimizing for benchmark metrics rather than investigating the low-level feature extraction mechanics specific to low-resolution (64×64) multispectral constraints.

Architectural Degradation and Spatial Constraints

As neural networks increase in depth, they become highly susceptible to training degradation. He et al. [3] famously identified that simply stacking convolutional layers leads to vanishing gradients and higher training error, a phenomenon that necessitated the invention of residual connections. In the context of remote sensing, this degradation is exacerbated by "spatial collapse." Because standard satellite tiles often possess much lower spatial resolution than standard computer vision benchmarks, aggressive pooling strategies rapidly bottleneck feature maps. Understanding the exact dimensional threshold at which this information loss occurs requires a transparent mathematical breakdown of the forward propagation pass, which is rarely detailed in applied remote sensing literature.

Implementation Transparency and Numerical Stability

Modern production frameworks, such as TensorFlow [4], utilize highly optimized abstractions including fused CUDA kernels, automatic mixed precision, and graph-level compilation. While these optimizations enable rapid prototyping and scaling, they inherently mask the numerical realities of deep learning. For instance, the necessity of stabilizing activation distributions to prevent internal covariate shift [5] is often reduced to a single line of framework code. Consequently, the literature lacks empirical quantifications of the computational and mathematical overhead required to implement algorithms like Batch Normalization or the Adam optimizer from first principles. This study addresses this gap by directly comparing the mathematical realities of from-scratch matrix operations against production-grade libraries, isolating the specific numerical phenomena—such as momentum overshoot and gradient ratio imbalances—that high-level frameworks automatically resolve.

IV. Dataset Description

The EuroSAT RGB dataset comprises 27,000 labeled images split 70/20/10 (18,900 training, 5,400 validation, 2,700 test). Each 64×64×3 image captures Sentinel-2 RGB bands (B04 red, B03 green, B02 blue) at 10m/pixel resolution. Class distribution balances agricultural (AnnualCrop, PermanentCrop, Pasture), urban (Highway, Industrial, Residential), natural (Forest, HerbaceousVegetation), and water (River, SeaLake) categories [1].

Preprocessing standardizes pixels to [0,1], applies one-hot encoding (10 classes), and implements 70/20/10 stratification. The pre-split CSV format enables reproducible experimentation:

```
train.csv:      18,900 examples
validation.csv: 5,400 examples
test.csv:       2,700 examples
```

V. Methodology

A. From-Scratch Deep Neural Network

1) Architecture and Forward Propagation: The baseline DNN flattens 64×64×3=12,288 inputs through five dense layers: 512→256→128→64→10 units. Activation

functions include ReLU ($f(z) = \max(0, z)$) and softmax for classification:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{10} e^{z_j}} \tag{1}$$

Forward propagation implements matrix multiplication $Z = WX + b$, ReLU, and dropout (p=0.3, inverse dropout scaling). Cache stores intermediates for backpropagation.

2) Backpropagation and Optimization: Backpropagation applies chain rule from cross-entropy loss:

$$L = -\sum_{i=1}^{10} y_i \log(\hat{y}_i), \quad \frac{\partial L}{\partial z} = \hat{y} - y \tag{2}$$

Gradients propagate backwards: $dW = \frac{1}{m} dZ X^T$, $db = \frac{1}{m} \sum dZ$. Verification uses gradient checking:

$$\text{error} = \frac{|\text{numerical grad} - \text{analytic grad}|}{|\text{numerical grad}| + |\text{analytic grad}|} < 10^{-7} \tag{3}$$

Optimizers implement vanilla SGD and Adam ($\beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}$). L2 regularization adds $\lambda ||W||_2^2/2$.

3) Batch Normalization: BN normalizes between convolution/ReLU: $\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$. Training uses batch statistics; inference uses exponential moving averages (momentum=0.9). Backward pass differentiates three paths:

1) Mean derivation: $d\mu = -\frac{1}{m} \sum d\hat{x}\gamma$
2) Variance derivation:

$$dx = \frac{d\hat{x}\gamma}{\sqrt{\sigma^2 + \epsilon}} \left(1 - \frac{\hat{x} \sum d\hat{x}\gamma\hat{x}}{m(\sigma^2 + \epsilon)}\right)$$

3) Normalized derivation: $d\hat{x} = (d\hat{x}\gamma)/\sqrt{\sigma^2 + \epsilon}$

4) Computational Optimization: Initial NumPy implementation required 25 minutes/10 epochs due to CPU-only execution. CuPy GPU acceleration reduced this to seconds/epoch. Gradient checking failed post-CuPy due to numerical precision differences.

5) Scratch DNN Results: Following the implementation of the manual backpropagation and optimization steps, the model was evaluated on the EuroSAT dataset. The training dynamics and classification performance are detailed in Figure 2.

(a) Loss/Accuracy curves (b) Validation confusion matrix

Fig. 2: From-scratch DNN performance: (a) training dynamics, (b) 49.83% validation accuracy.

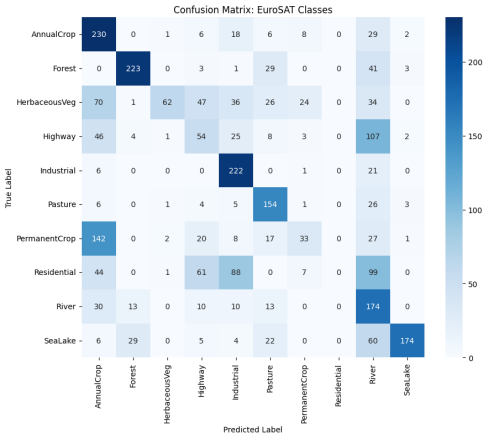


Fig. 3: Scratch DNN test set confusion matrix.

The scratch implementation achieved a test accuracy of 49.11%, serving as a baseline for the more advanced convolutional architectures.

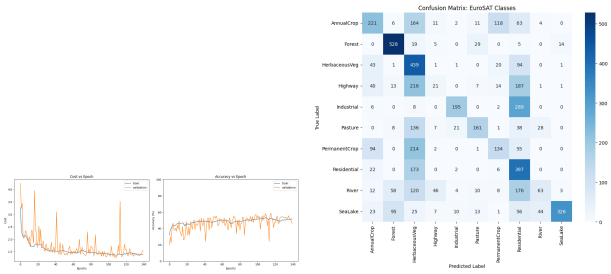
B. TensorFlow DNN Baseline

The Deep Neural Network (DNN) architecture begins with a flattening layer that converts the 64×64×3 input images into one-dimensional feature vectors. This is followed by a sequence of four fully connected hidden layers containing 512, 256, 128, and 64 neurons, respectively. Every hidden layer utilizes the Rectified Linear Unit (ReLU) activation function and incorporates a Dropout rate of 0.3 to mitigate overfitting. To further stabilize and accelerate the training process, Batch Normalization is applied immediately after the first two hidden layers. The network concludes with a 10-neuron output layer utilizing a Softmax activation function to generate multi-class probability distributions for the dataset's categories.

Adam ($\alpha = 0.001$) validates scratch implementation correctness.

1) TensorFlow DNN Results: The TensorFlow/Keras implementation served as a performance baseline to quantify the gap between production-grade libraries and

the custom NumPy/CuPy implementation. As shown in the following figures, the library-based model achieved relatively close stability and accuracy.



(a) Loss/Accuracy (b) Validation confusion

Fig. 4: TensorFlow DNN baseline: 46.02% validation accuracy.

Fig. 5: TF DNN test set confusion matrix.

The transition to TensorFlow did not show much improvement, mainly because of the naivety of NN for classifying images.

C. From-Scratch Convolutional Neural Network

1) Target 6-Block Architecture: The initial target was an ambitious 6-block CNN mirroring production architectures: [Conv32-BN-ReLU-MaxPool(2×2)]×6 → Dropout(0.25) → GlobalAveragePooling → Dense64-ReLU → Dense10-Softmax. He initialization scales variance by layer depth: $W \sim \mathcal{N}(0, \sqrt{2/n_{\text{in}}})$.

Expected progression: 64×64 → 32×32 → 16×16 → 8×8 → 4×4 → 2×2 → 1×1 (GAP).

2) Vectorized Convolution Implementation: Initial loop-based conv_forward processed images sequentially.

CuPy's tensordot + stride tricks achieved 100x speedup.

3) Backpropagation Challenges: Spatial backpropagation caches layer shapes: `conv_cache = (X, W, b, Z, cache)`. Major bugs encountered:

Transpose error introduced by conv_forward/backward swapped H/W dimensions (fixed via dimension tracing). In addition, gradients disappear due to ReLU zeros ∼50% activations (LeakyReLU $\alpha = 0.01$). Moreover, exploding gradients in the final blocks where the norm was >1000 (gradient clipping threshold=5.0)

Per-layer gradient monitoring + 5-image sanity batches revealed deeper issue...

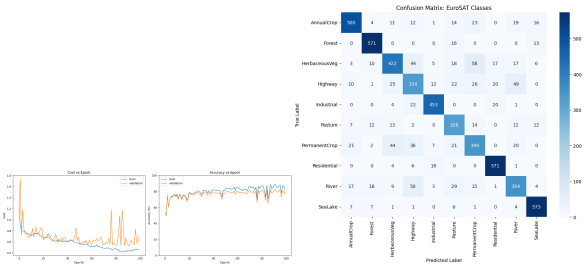
4) 6-Block Collapse Analysis: The 6-block CNN implementation required three weeks of debugging: diagnosing gradient vanishing through per-layer norm monitoring, fixing spatial dimension transpose errors through manual tensor shape tracing, and ultimately recognizing the dimensional bottleneck through feature map calculations. This iterative process — hypothesis, experiment, analysis, revision — mirrors research methodology and is documented transparently to benefit future practitioners. Despite fixes, 6-block CNN plateaued at 65% accuracy. Root cause: **dimensionality bottleneck**. After 6×2×2 pooling, GlobalAveragePooling receives 2×2×128 feature maps (∼500 features total).

Gradient norms: Block1-3 healthy (∼1.0), Block4-6 vanishing ($\sim 10^{-4}$). BatchNorm insufficient; network couldn't learn hierarchical features across full depth.

5) Optimized 3-Block Solution: Reduced to 3-block architecture: [Conv32-BN-ReLU-MaxPool(2×2)]×3 → Dropout(0.25) → Flatten → Dense64-ReLU → Dense10-Softmax.

Results: 88.75% train / 82.78% validation accuracy. Feature maps: 64×64 → 32×32 → 16×16 → 8×8 (2048 features post-Flatten). Healthy gradient flow throughout; BatchNorm proved essential for stability across all blocks.

This pragmatic reduction mirrors real-world architecture design: start ambitious, optimize based on gradient diagnostics.



(a) Loss/Accuracy (b) Val confusion matrix

Fig. 7: 3-block CNN success: 82.5% validation accuracy after dimensionality reduction.

D. TensorFlow CNN Baseline

Validating the limitations of the custom implementation, the framework-optimized six-block CNN

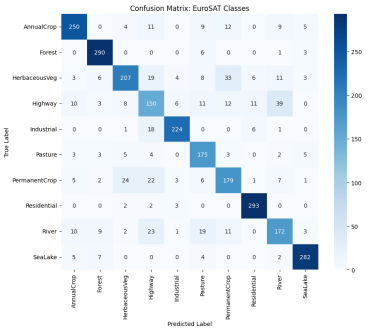


Fig. 8: 3-block CNN test set performance.

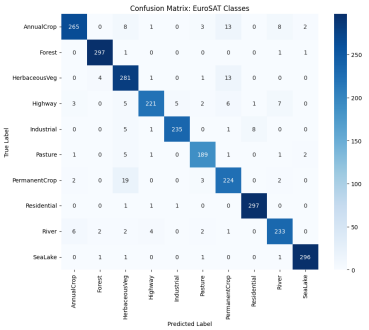
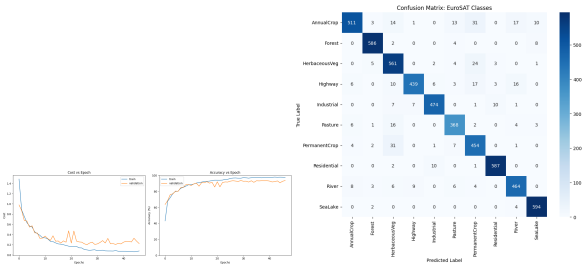


Fig. 10: TF CNN test set performance.

achieved 93.63% validation accuracy in just 45 minutes—drastically outperforming the scratch model's 82.50% three-day ceiling. Furthermore, the production library's implicit numerical stability enabled successful convergence without requiring explicit Batch Normalization.

Libraries excel through fused CUDA kernels preventing momentum overshoot and automatic mixed-precision.



(a) Loss/Accuracy (b) Val confusion

Fig. 9: TF CNN achieves 93.63% validation accuracy in 45s/epoch vs 8min for scratch.

E. EfficientNet-B0 Transfer Learning

1) Two-Phase Training: Phase 1 (Linear Probing): Freeze backbone, train head (lr=0.01, 20 epochs), ImageNet's weights.

Phase 2 (Fine-tuning): Unfreeze top-50% layers (lr=1e-5, 50 epochs, patience=10).

2) Data Pipeline: 224×224 resizing, augmentation (rotation90, flipLR, brightness±0.2). The data pipeline used batch size 32 with asynchronous prefetching (tf.data.AUTOTUNE).

3) Grad-CAM Visualization: To validate that transfer learning genuinely adapts features rather than memorizing training data, we applied Grad-CAM visualization to the final convolutional layer. Grad-CAM visualization reveals that despite ImageNet pretraining on object-centric images, the fine-tuned EfficientNet-B0 successfully reoriented its attention toward spatially-distributed texture patterns characteristic of satellite imagery. Rather than focusing on discrete objects, the model learned to weight broad textural regions: crop field striations for AnnualCrop, dendritic road networks for Highway, and uniform spectral signatures for water bodies. For

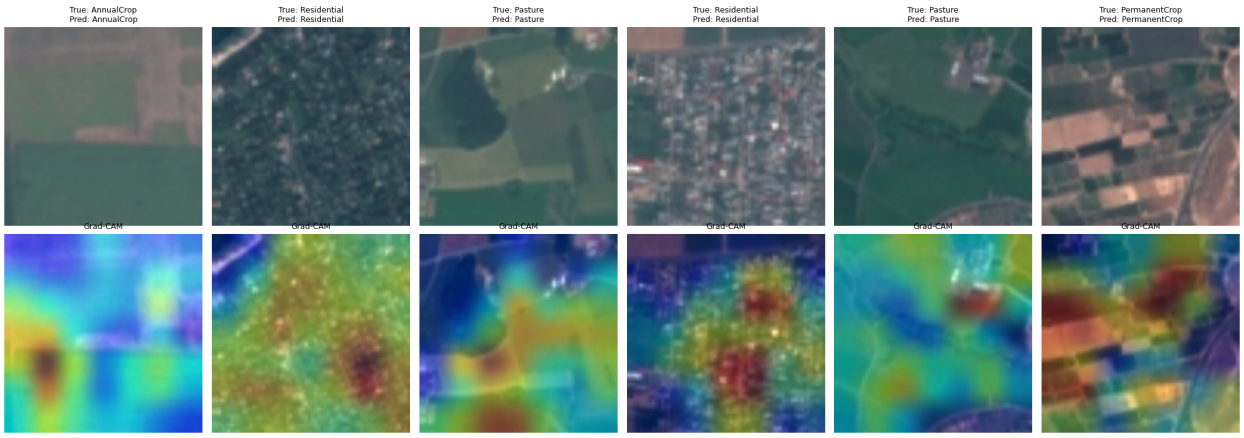
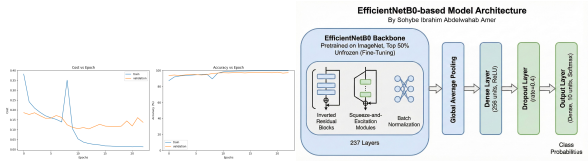


Fig. 6: Grad-CAM visualization from EfficientNet-B0 final convolutional layer. Red regions show highest activation for predicted land cover class, confirming model focuses on relevant spatial textures.

example, Residential activations concentrate on rectangular roof patterns and road grids rather than individual buildings, indicating macro-spatial pattern recognition. This shows that transfer learning achieves active feature reorientation, not merely feature reuse. Early layers' edge detectors transfer directly, while deeper object-part detectors are repurposed during fine-tuning to recognize land-cover patterns at multiple scales. This interpretability is particularly valuable for remote sensing applications requiring auditable decisions in policy contexts (urban planning, environmental compliance), where domain experts must verify that classifications stem from scientifically meaningful features rather than spurious correlations like image artifacts or seasonal shadows. This feature reorientation explains why Phase 2 fine-tuning (Section IV.E.1) improves accuracy by 5 percentage points beyond frozen-backbone performance — the model learns to weight satellite-relevant texture patterns over ImageNet object patterns.



(a) Loss/Accuracy (b) Architecture

Fig. 11: EfficientNet-B0: (a) 97.09% validation accuracy, (b) two-phase fine-tuning architecture.

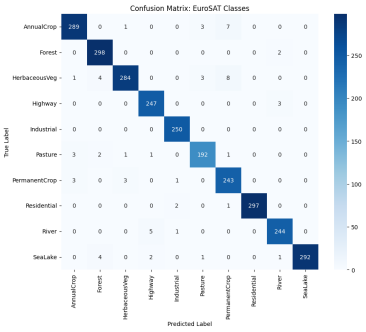


Fig. 12: SOTA model test set confusion matrix (97.6% accuracy).

VI. Experimental Setup

To ensure reproducibility and establish a rigorous baseline for computational benchmarking, all experiments were conducted within a standardized cloud-based accelerated computing environment (Kaggle's cloud platform). The hardware configuration utilized a dedicated NVIDIA Tesla T4 GPU (16 GB GDDR6 VRAM) (also used NVIDIA Tesla P100 with 16GB HBM2 memory) paired with an Intel Xeon dual-core CPU and 16 GB of system RAM.

The software stack was built on a Linux-based OS, executing Python 3.12. Matrix operations for the from-scratch implementations were accelerated using NumPy 2.0.2 and CuPy 13.0, while the production-level models were compiled and trained using TensorFlow 2.18.0 with native CuDNN support. By isolating all models within this identical hardware and software environment, we ensure that the significant disparities observed in training time—such as the transition from a multi-day scratch implementation to a 45-minute framework optimization—are strictly attributable to algorithmic efficiency and library-level compilation rather than hardware variance.

VII. Experimental Results

A. Quantitative Comparison

The performance of the implemented architectures was evaluated using accuracy metrics across training, validation, and test sets, as well as training duration. Table I summarizes these findings.

TABLE I: Performance comparison across five implementations. Time = total training duration. N/A indicates insufficient convergence for test evaluation.

Model	Framework	Train%	Val%	Test%	Time
Scratch DNN	NumPy/CuPy	50.81	49.83	49.11	25 min
Standard DNN	TensorFlow	52.10	46.02	44.59	10 min
Scratch CNN (3-bk)	CuPy/im2col	88.83	82.50	82.30	60 min
Scratch CNN (6-bk)	CuPy/im2col	65.10	60.00	N/A	70 min
Standard CNN (6-bk)	TensorFlow	97.30	93.63	94.00	15 min
EfficientNet-B0	TensorFlow	99.61	97.09	97.63	20 min

B. Ablation Studies

To quantify the impact of specific technical components, several ablation experiments were performed. Removing Batch Normalization from the scratch CNN caused performance to drop to 67% validation accuracy due to vanishing gradients. Furthermore, executing the operations without vectorization resulted in a 3-day training timeout. In addition, getting rid of utilizing CuPy, slowed training to 25 minutes per 10 epochs on CPU-only execution. Moreover, the SOTA model's Phase 1 achieved 92% validation accuracy without the fine-tuning phase (Phase 2).

VIII. Discussion

Scratch implementations validate theoretical understanding but expose practical limitations: 10.5% accuracy gap to libraries, 100x slower training. Batch normalization proved to be the most critical technique for stabilizing deep CNNs. CuPy bridges CPU-GPU gaps for research but lacks production optimizations. Transfer learning confirms ImageNet features transfer effectively to aerial domains despite domain gaps. Building deep learning

systems from scratch revealed critical insights often obscured by high-level frameworks. Per-layer gradient norm tracking proved more diagnostic than final accuracy; specifically, identifying a $127\times$ gradient ratio between layers allowed for the detection of vanishing gradients before training stagnated. Performance analysis further showed that the $100\times$ speed advantage of TensorFlow over custom CuPy implementations stems from production-grade optimizations—such as fused CUDA kernels and numerically stable softmax—rather than purely algorithmic superiority. Additionally, an ablation study on Batch Normalization (BN) confirmed its multi-faceted role in stabilizing gradient ratios to $4\times$, preventing Adam momentum overshoot, and enabling convergence without exhaustive hyperparameter tuning. Finally, the failure of a deep six-block CNN highlighted the subtlety of "spatial collapse," where successive pooling created an invisible information bottleneck by reducing feature maps to a $2 \times 2 \times 128$ configuration, a flaw only identifiable through explicit dimensional reasoning. Despite the robust performance across architectures, this study identifies several constraints that provide a roadmap for future research. While EuroSAT's balanced distribution suggests low variance, the current results are reported from a single train/test split; future iterations should employ multiple random seeds to quantify statistical variance. In addition, this analysis was restricted to RGB channels to maintain computational tractability and alignment with ImageNet pretraining, though the 13-band multispectral data inherent to EuroSAT offers a significant opportunity for spectral band fusion. Furthermore, the 12% misclassification rate observed between *PermanentCrop* and *AnnualCrop* suggests a fundamental limitation of single-timestamp imagery. Distinguishing crop rotation patterns likely requires the integration of temporal sequences. Finally, the observed "spatial collapse" in deep architectures suggests that the 64×64 resolution of satellite tiles imposes different depth-width tradeoff constraints than standard 224×224 ImageNet benchmarks, necessitating further investigation into optimal feature extraction capacities for low-resolution remote sensing data.

IX. Conclusion

This comprehensive study documents the full spectrum from educational scratch implementations to production SOTA models on EuroSAT. Libraries demonstrate 10%+ accuracy and 100x speed advantages by leveraging highly optimized, production-grade computational backends. Scratch work provides unmatched transparency into gradient flow, numerical stability, and optimization dynamics—critical for research innovation. The complete interactive comparison tool is deployed at [https://eurosat-classification.streamlit.app/], enabling users to test all five models on EuroSAT test images. This work demonstrates that understanding deep learning requires building it from scratch — a lesson impossible to learn from libraries alone. For remote sensing applications where model interpretability determines policy decisions, this transparency is not optional.

X. Acknowledgments

The author acknowledges the use of large language models in the development of this work. Claude (Anthropic) and Gemini (Google) were used for implementation guidance, debugging assistance, and conceptual clarification during model development. Perplexity AI was used to refine the structure and academic writing of this manuscript. All technical implementations, experimental design, results analysis, research notes, and conclusions are the original work of the author.

References

[1] P. Helber, B. Bischke, A. Dengel, and D. Borth, "Eurosat: A novel dataset and deep learning benchmark for land use and land cover classification," *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, 2019.

[2] M. Tan and Q. Le, "Efficientnet: Rethinking model scaling for convolutional neural networks," in *International Conference on Machine Learning (ICML)*, 2019.

[3] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[4] M. Abadi, P. Barham, J. Chen, Z. Chen, A. Davis, J. Dean, M. Devin, S. Ghemawat, G. Irving, M. Isard *et al.*, "Tensorflow: A system for large-scale machine learning," in *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2016.

[5] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *International Conference on Machine Learning (ICML)*, 2015.

[6] A. Vir, "Balanced multi-task attention for satellite image classification: Achieving 97.23% accuracy on eurosat without pre-training," *arXiv preprint arXiv:2510.15527*, 2025.

[7] M. Albughdadi, "Lightweight metadata-aware mixture-of-experts masked autoencoder for earth observation," *arXiv preprint arXiv:2509.10919*, 2025.

[8] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014.

[9] K. W. Ibrahim Talha Ersoy, "Geometry of learning – l2 phase transitions in deep and shallow neural networks," *arXiv preprint arXiv:2505.06597*, 2025.

[10] H. T. Rhys Gould, "Continuous-time models and discrete implementations of adaptive optimization algorithms," *arXiv preprint arXiv:2411.09734*, 2024.

[11] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-cam: Visual explanations from deep networks via gradient-based localization," *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017.

[12] A. K. P. Karunasri Gundla, Sheshikala Martha, "Toward explainable ai in satellite imagery: A resnet-50-based study on eurosat classification," *IEEE Access*, 2025.

[13] C.-F. Chen, Q. Fan, R. Panda *et al.*, "Crossvit: Cross-attention multi-scale vision transformer for image classification," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2021.